

# Module 1: The Dynamic DOM & Event Delegation

## Abstract

**Objective:** Understand why code often “breaks” when adding new elements to a page dynamically, and learn how to fix it using the *Event Delegation* pattern.

---

## 1 Part 1: Theory Reference

Before writing code, it is essential to understand how the browser handles user interaction. It is not as simple as “Button clicked → Function runs.”

### 1.1 The DOM Hierarchy

Everything on a webpage is a node in a tree structure. To manipulate the page effectively, you must understand the relationships:

- **Parent:** The container holding an element.
- **Child:** The element inside the container.
- **Sibling:** Elements sharing the same parent.

Key Method: `.closest()`

```
element.closest('.class-name')
```

This method looks **UP** the tree (at parents and grandparents) to find the nearest container matching the selector. It is crucial for finding which “Row” a clicked button belongs to.

## 1.2 The Invisible Flow: Event Bubbling

When you click a button inside a `<div>`, you aren't just clicking the button. You are clicking the Button, *then* the Div, *then* the Body, *then* the HTML tag.

- **Bubbling:** The event starts at the **Target** (what you clicked) and bubbles up to the root.
- **Why this matters:** We can use this to our advantage. Instead of putting a “listener” on every single child element, we can put one listener on the container to “catch” bubbles coming from its children.

## 1.3 The “Ghost Element” Problem

This is the most common bug for beginners. Consider the following code:

```
1 // THE WRONG WAY
2 const buttons = document.querySelectorAll('.delete-btn');
3
4 buttons.forEach(btn => {
5     btn.addEventListener('click', deleteItem);
6 });
```

Listing 1: The Wrong Way

**Why it fails:**

1. This code runs **once** when the page loads.
2. It finds only the buttons that exist *at that moment*.
3. If you add a new button later via JavaScript, it does not have this listener attached. It is a “Ghost”—it looks real, but it is not interactive.

## 1.4 The Solution: Event Delegation

Instead of attaching a listener to every child, attach one listener to the parent.

**The Pattern:**

1. Attach **one** listener to the parent container (e.g., `<ul>` or `<div id="list">`).
2. When a click happens, check `event.target` to see *what* started the bubble.
3. If the target matches our button class, run the code.

## 2 Part 2: The Exercise

### 2.1 Project: The “Broken” Task Manager

**Scenario:** You have joined a team working on a Task Manager app. The previous developer built a “Remove Task” feature, but it has a critical bug: **It works for the initial tasks, but fails for any new tasks you create.**

### 2.2 Step 1: Setup Files

Create a folder with the following three files.

#### 1. index.html

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <link rel="stylesheet" href="style.css">
5 </head>
6 <body>
7   <div id="app">
8     <h1>Task Manager</h1>
9     <div class="input-group">
10      <input type="text" id="taskInput" placeholder="New Task..."
11    >
12      <button id="addBtn">Add</button>
13    </div>
14    <ul id="taskList">
15      <li class="task-item">
16        <span>Buy Milk</span>
17        <button class="delete-btn">X</button>
18      </li>
19      <li class="task-item">
20        <span>Walk Dog</span>
21        <button class="delete-btn">X</button>
22      </li>
23    </ul>
24  </div>
25  <script src="script.js"></script>
26 </body>
27 </html>
```

## 2. style.css

```
1 body { font-family: sans-serif; padding: 2rem; }
2 #taskList { list-style: none; padding: 0; }
3 .task-item {
4     background: #f4f4f4;
5     border-bottom: 1px solid #ddd;
6     padding: 10px;
7     display: flex;
8     justify-content: space-between;
9 }
10 .delete-btn { background: red; color: white; border: none; cursor:
    pointer; }
```

## 3. script.js (The Buggy Code)

**Copy this exactly.** This represents the legacy code you must fix.

```
1 const addBtn = document.getElementById('addBtn');
2 const taskList = document.getElementById('taskList');
3 const input = document.getElementById('taskInput');
4
5 // 1. Add New Task Functionality
6 addBtn.addEventListener('click', () => {
7     const newTask = document.createElement('li');
8     newTask.className = 'task-item';
9     newTask.innerHTML = `
10         <span>${input.value}</span>
11         <button class="delete-btn">X</button>
12     `;
13     taskList.appendChild(newTask);
14     input.value = '';
15 });
16
17 // 2. THE BUGGY DELETE FUNCTIONALITY
18 // This only finds buttons that exist on load!
19 const deleteButtons = document.querySelectorAll('.delete-btn');
20
21 deleteButtons.forEach(btn => {
22     btn.addEventListener('click', (e) => {
23         // e.target is the button. parentElement is the <li>
24         e.target.parentElement.remove();
25     });
26 });
```

## 2.3 Step 2: Your Mission

Fix the code so that **newly added tasks** can also be deleted.

### Strict Constraints

1. You are **NOT** allowed to add a new event listener inside the `addBtn` function.
2. You are **NOT** allowed to use `onclick="..."` in the HTML.
3. You **MUST** delete the entire “Buggy Delete Functionality” block and replace it with **Event Delegation**.
4. You **MUST** attach your listener to `#taskList`.

# Module 2: Async/Await & API Error Handling

## Abstract

**Objective:** Learn how to fetch data from real-world APIs, handle “Loading” states gracefully, and prevent application crashes using `try/catch` and `response.ok`.

---

## 1 Part 1: Theory Reference

JavaScript is single-threaded (it does one thing at a time). However, fetching data from a server takes time (0.5s - 2s). We cannot freeze the entire page while waiting.

### 1.1 The Modern Syntax: Async / Await

In the past, we used “Callbacks” or “.then() chains”. Modern JavaScript uses keywords that make asynchronous code look synchronous.

- **async:** Declares that a function handles time-consuming operations.
- **await:** Pauses the execution of *that specific function* (not the whole page) until data arrives.

### 1.2 The Common Trap: Silent Failures

A major misconception is that ‘fetch’ will throw an error if the server says “404 Not Found”. **It does not.**

#### The Fetch Rule

`fetch()` only rejects (throws an error) if the **network fails** (e.g., no internet). If the server returns a 404 or 500 error, `fetch()` considers this a “successful” communication. You must manually check `response.ok`.

## 1.3 The Pattern: Try / Catch / Finally

Robust code handles three states:

1. **Loading:** Tell the user we are working.
2. **Success:** Display the data.
3. **Error:** Tell the user something went wrong (don't just `console.log`).

```
1 async function getData() {  
2   try {  
3     // 1. Start Loading  
4     loader.style.display = 'block';  
5  
6     const response = await fetch(url);  
7  
8     // 2. Manual Error Check  
9     if (!response.ok) {  
10      throw new Error('Server Error: ${response.status}');  
11    }  
12  
13    const data = await response.json();  
14    renderData(data);  
15  
16  } catch (error) {  
17    // 3. Handle Error (Network or 404)  
18    alert(error.message);  
19  } finally {  
20    // 4. Cleanup (Always runs)  
21    loader.style.display = 'none';  
22  }  
23 }
```

Listing 1: Robust Fetch Pattern

## 2 Part 2: The Exercise

### 2.1 Project: The “Fragile” User Viewer

**Scenario:** You are building a dashboard that pulls user profiles from a database. The previous developer wrote a script that assumes everything always works perfectly.

**The Problem:**

1. If the internet is slow, the user sees nothing (no loading indicator).
2. If the user ID doesn't exist (404), the app crashes silently or shows “undefined”.

### 2.2 Step 1: Setup Files

Create a new folder with these files.

#### 1. index.html

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <link rel="stylesheet" href="style.css">
5 </head>
6 <body>
7   <div class="card">
8     <h2>User Viewer</h2>
9     <div class="controls">
10      <input type="number" id="userIdInput" value="1" placeholder
      ="Enter ID">
11      <button id="loadBtn">Load User</button>
12    </div>
13
14    <div id="userCard" class="hidden">
15      <h3 id="userName">Name Here</h3>
16      <p id="userEmail">email@example.com</p>
17    </div>
18
19    <p id="loadingMsg" class="hidden">Fetching data...</p>
20
21    <p id="errorMsg" class="hidden error-text"></p>
22  </div>
23  <script src="script.js"></script>
24 </body>
25 </html>
```

## 2. style.css

```
1 body { font-family: sans-serif; display: flex; justify-content: center;
  padding: 50px; }
2 .card { border: 1px solid #ccc; padding: 20px; border-radius: 8px;
  width: 300px; }
3 .hidden { display: none; }
4 .error-text { color: red; font-weight: bold; }
5 input { padding: 5px; width: 60%; }
6 button { padding: 5px 10px; cursor: pointer; }
```

## 3. script.js (The Fragile Code)

**Copy this exactly.** This code works for valid users but breaks easily.

```
1 const loadBtn = document.getElementById('loadBtn');
2 const userIdInput = document.getElementById('userIdInput');
3 const userCard = document.getElementById('userCard');
4 const userName = document.getElementById('userName');
5 const userEmail = document.getElementById('userEmail');
6
7 // THE BUGGY FUNCTION
8 // - No Error Handling
9 // - No Loading State
10 // - No "Not Found" check
11
12 loadBtn.addEventListener('click', async () => {
13   const userId = userIdInput.value;
14
15   // 1. Fetch data
16   const response = await fetch('https://jsonplaceholder.typicode.com/
  users/${userId}');
17
18   // 2. Parse JSON
19   const data = await response.json();
20
21   // 3. Update UI
22   // If user is 404, 'data' is empty, but code tries to read data.
  name -> Undefined!
23   userCard.classList.remove('hidden');
24   userName.textContent = data.name;
25   userEmail.textContent = data.email;
26 });
```

## 2.3 Step 2: Your Mission

Refactor the code to handle real-world imperfection.

### Strict Constraints

1. You **MUST** show the `#loadingMsg` element immediately when the button is clicked, and hide it when data arrives.
2. You **MUST** use a `try / catch` block.
3. You **MUST** check `if (!response.ok)` to handle 404 errors (User not found).
4. If an error occurs, you must hide the user card and show the `#errorMsg` with text: "User not found!"

## 2.4 Step 3: Test Cases

- **Input 1:** Should show "Leanne Graham".
- **Input 9999:** Should show "User not found!" (Currently shows undefined).
- **Network Test:** (Chrome DevTools → Network → Offline). Should show "Network Error".